

UPI Fraud Detection System

Detecting suspicious digital payment transactions with Machine Learning

PYTHON • SQL • MACHINE LEARNING • FASTAPI • STREAMLIT

Prepared by: Mohd. Razi Khan

Project Overview

This project detects likely-fraudulent UPI transactions using machine learning, combining a supervised classifier with an unsupervised anomaly detector, backed by a SQL database and served through a REST API and an interactive dashboard.

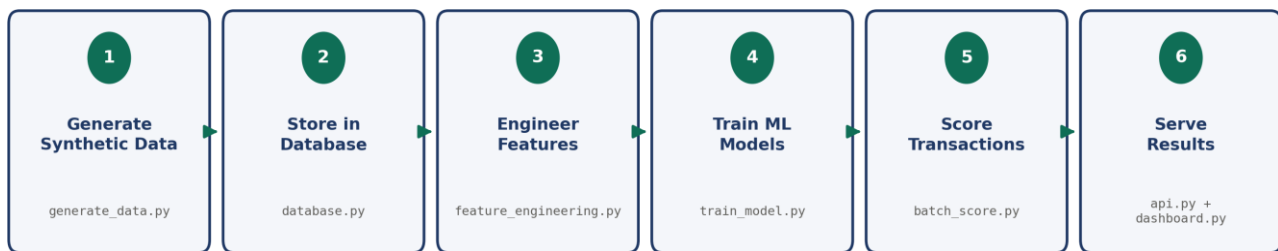
It demonstrates three skills together in one working system: **Python** (data pipeline, backend, machine learning), **SQL** (schema design, storing and querying transaction data), and **AI/ML** (imbalanced classification, feature engineering, model evaluation).

Note: real UPI transaction data is private and regulated, so this project uses realistic synthetic data with deliberately engineered fraud patterns (odd-hour transactions, unusually high amounts, new/unrecognized devices) — the standard approach for a project of this kind.

Tech Stack

Layer	Tool	Purpose
Data generation	Faker	Creates realistic synthetic users & transactions
Database	SQLite + SQLAlchemy	Stores users, transactions, flagged records
Feature engineering	pandas	Converts raw data into numeric ML features
Supervised model	Random Forest (scikit-learn)	Learns known fraud patterns from labels
Class balancing	SMOTE (imbalanced-learn)	Fixes the rare-fraud-event imbalance
Anomaly detection	Isolation Forest (scikit-learn)	Flags unusual patterns without labels
Backend API	FastAPI	Real-time transaction scoring endpoint
Dashboard	Streamlit	Charts, flagged list, live transaction tester

Project Workflow



Each stage's output feeds the next: synthetic data is generated and stored in SQL, transformed into features, used to train two complementary models, applied to score every transaction, and finally served through the API and dashboard for real-time use.

How Fraud Detection Works

Features used per transaction: amount, deviation from the sender's personal average amount, new-device flag, odd-hour flag (midnight–5 AM or 11 PM), high-amount flag (top 10%), and hour of day.

Random Forest (supervised) — 200 decision trees trained on SMOTE-balanced data. Since only ~5% of transactions are fraud, SMOTE generates synthetic minority-class examples so the model actually learns to recognize fraud rather than simply predicting “not fraud” every time.

Isolation Forest (unsupervised) — doesn't use fraud labels at all. It isolates unusual transactions using random splits, acting as a safety net for patterns the labeled data didn't cover.

Final risk score formula:

```
risk_score = 0.7 × RandomForest_probability + 0.3 × anomaly_flag
```

A transaction is flagged as fraud when **risk_score** ≥ **0.5**.

Model Results

Evaluated on a held-out test set of 1,000 transactions the model had never seen:

Metric	Score	Meaning
Precision	1.00	Every transaction flagged as fraud actually was fraud
Recall	1.00	Every actual fraud case in the test set was caught
F1-score	1.00	Combined precision/recall measure — perfect here

These scores are unusually high because the synthetic data has deliberately distinct fraud patterns. Real-world fraud is noisier and would realistically show lower scores — worth stating honestly when presenting this project.

Key Talking Points

Why synthetic data instead of real UPI data?

Real transaction data is private and regulated (banking data, PII). Synthetic data with realistic fraud patterns is the standard, expected approach for a project like this.

Why SMOTE?

Fraud is a rare-event problem. Without balancing, a model can reach high accuracy by simply predicting “not fraud” every time. SMOTE generates synthetic fraud examples so the model learns real patterns.

Why precision/recall instead of accuracy alone?

With imbalanced classes, accuracy is misleading. Recall matters most — missing real fraud is usually costlier than a false alarm.

Why two models instead of one?

Random Forest is strong on patterns it was trained on. Isolation Forest needs no labels, so it can catch new, unseen fraud patterns — a safety net against blind spots.

What would you improve for production?

Use real anonymized data, add behavioral features like transaction velocity and account-graph relationships, retrain periodically, and add authentication/rate-limiting to the API.

Limitations & Future Improvements

- Synthetic data has clean, separable fraud patterns; real-world fraud is noisier
- No transaction-velocity features (multiple transactions in a short window)
- No graph-based features (detecting rings of connected accounts)
- No authentication layer on the API
- No automatic retraining pipeline as fraud patterns evolve

Next steps: deploy the API and dashboard online (Render/Railway), add a login layer, add velocity/graph features, and connect to a real anonymized dataset if one becomes available.

Project links: GitHub repository — [<https://github.com/Razikhan786/UPI-fraud-detection.git>] | Built as a portfolio project demonstrating applied Python, SQL, and Machine Learning skills.